Aula 1 - Fundamentos de Seguranca da Informacao

Confidencialidade, Integridade e Disponibilidade formam a triade CIA. A seguranca procura reduzir risco: uma ameaca explora uma vulnerabilidade e pode causar impacto. CVE identifica vulnerabilidades publicas, CVSS ajuda a estimar severidade e CWE classifica tipos de fraqueza.

Pontos-chave

- Diferencie vulnerabilidade, exploit e payload.
- Use menor privilegio e defesa em profundidade.
- Seguranca e gestao de risco, nao promessa de invulnerabilidade.

Exemplo tecnico

Codigo / comando

```
function risco(probabilidade, impacto) {
  return probabilidade * impacto;
}
console.log(risco(3, 4));
```

Portugol - equivalente didatico

```
funcao inteiro risco(inteiro probabilidade, inteiro impacto)
    retorne probabilidade * impacto
fimfuncao

escreva(risco(3, 4))
```

Aula 2 - HTTP, Requests, Responses e Parametros

HTTP organiza a conversa entre cliente e servidor. Uma requisicao tem metodo, caminho, headers e eventualmente body. A resposta possui status, headers e body. Parametros podem aparecer na URL, no corpo ou em headers, mas nunca devem ser considerados confiaveis apenas por terem vindo do frontend.

Pontos-chave

- GET consulta; POST normalmente envia/cria; PATCH altera parcialmente; DELETE remove.
- 200, 301, 400, 401, 403, 404 e 500 sao status comuns.
- Frontend nao substitui validacao no backend.

Exemplo tecnico

Codigo / comando

```
app.get("/produto", (req, res) => {
  const id = req.query.id;
  res.json({ id });
});
```

Portugol - equivalente didatico

```
cadeia id
id <- parametroURL("id")
responderJSON("id", id)
```

Aula 3 - Autenticacao e Autorizacao

Autenticacao responde quem e o usuario; autorizacao responde o que ele pode fazer. Um sistema pode autenticar corretamente e ainda possuir Broken Access Control se nao verificar permissoes em cada recurso.

Pontos-chave

- 401: identidade ausente/invalida em muitos cenarios.
- 403: identidade conhecida, mas sem permissao.
- Autorizacao precisa acontecer no servidor.

Exemplo tecnico

Codigo / comando

```
app.get("/admin", autenticar, (req,res) => {
  if (req.user.role !== "admin") return res.sendStatus(403);
  res.send("Area administrativa");
});
```

Portugol - equivalente didatico

```
se nao autenticado entao
    responder(401)
senao se papel <> "admin" entao
    responder(403)
senao
    escreva("Area administrativa")
fimse
```

Aula 4 - Senhas, Hash, Salt e Bcrypt

Senha nao deve ser armazenada em texto puro. Funcoes de hash de senha como bcrypt usam custo computacional e salt para dificultar ataques contra bancos vazados.

Pontos-chave

- Hash nao e criptografia reversivel.
- Salt evita que senhas iguais produzam o mesmo resultado de forma trivial.
- Use bibliotecas maduras, nao crie algoritmo proprio.

Exemplo tecnico

Codigo / comando

```
const bcrypt = require("bcrypt");
const hash = await bcrypt.hash(password, 12);
const ok = await bcrypt.compare(password, hash);
```

Portugol - equivalente didatico

```
hash <- gerarHashSenha(senha, 12)
ok <- compararSenha(senha, hash)
se ok entao escreva("Senha valida") fimse
```

Aula 5 - Sessoes, Cookies e JWT

Depois do login, o sistema precisa manter estado. Isso pode ocorrer por sessao no servidor identificada por cookie ou por tokens como JWT. Cookies sensiveis devem considerar HttpOnly, Secure e SameSite conforme o contexto.

Pontos-chave

- Cookie e um mecanismo de transporte/armazenamento no cliente.
- JWT assinado nao deve ser apenas decodificado: precisa ser verificado.
- Tokens devem expirar e ser protegidos.

Exemplo tecnico

Codigo / comando

```
const payload = jwt.verify(token, process.env.ACCESS_SECRET);
req.user = payload;
```

Portugol - equivalente didatico

```
dados <- verificarToken(token, SEGREDO)
usuarioAtual <- dados
```

Aula 6 - IDOR e Broken Access Control

IDOR acontece quando um identificador controlado pelo usuario e usado para acessar um objeto sem verificar se aquele usuario pode acessar o objeto. A correcao nao e esconder o ID: e validar propriedade/permissao.

Pontos-chave

- IDs previsiveis nao sao a causa principal.
- O backend deve combinar objeto + identidade autenticada.
- Teste apenas em laboratorio ou escopo autorizado.

Exemplo tecnico

Codigo / comando

```
const pedido = await db.pedidos.findById(req.params.id);
if (pedido.userId !== req.user.id) return res.sendStatus(403);
res.json(pedido);
```

Portugol - equivalente didatico

```
pedido <- buscarPedido(id)
se pedido.usuarioId <> usuarioAtual.id entao
    responder(403)
senao
    responderJSON(pedido)
fimse
```

Aula 7 - SQL Injection

SQL Injection surge quando dados nao confiaveis sao concatenados ao comando SQL. A defesa principal e separar instrucao de dados por queries parametrizadas, alem de validar entradas e usar menor privilegio no banco.

Pontos-chave

- Nunca monte SQL critico por concatenacao de entrada.
- ORM nao elimina automaticamente todo risco.
- Mensagens de erro detalhadas nao devem vazar internamente.

Exemplo tecnico

Codigo / comando

```
const r = await db.query(
  "SELECT * FROM users WHERE email = $1",
  [email]
);
```

Portugol - equivalente didatico

```
consulta <- "SELECT * FROM users WHERE email = ?"
resultado <- executarSQL(consulta, email)
```

Aula 8 - Cross-Site Scripting - XSS

XSS ocorre quando conteudo controlado por usuario vira codigo executavel no navegador de outra pessoa. Context-aware output encoding e frameworks que escapam por padrao ajudam, mas insercoes perigosas de HTML continuam exigindo cuidado.

Pontos-chave

- Stored, Reflected e DOM XSS sao categorias comuns.
- Nao confie apenas em filtros de palavras.
- CSP e camada adicional, nao substitui correcao.

Exemplo tecnico

Codigo / comando

```
function escaparHTML(s) {
  return s.replace(/[&<>"']/g, c => MAP[c]);
}
res.send(`<p>${escaparHTML(nome)}</p>`);
```

Portugol - equivalente didatico

```
nomeSeguro <- escaparHTML(nome)
escreva("<p>" + nomeSeguro + "</p>")
```

Aula 9 - CSRF

CSRF explora o fato de o navegador enviar automaticamente credenciais como cookies para um site. Para operacoes sensiveis, use tokens anti-CSRF quando aplicavel, SameSite e validacao de origem/contexto.

Pontos-chave

- O risco e maior quando autenticacao depende de cookies enviados automaticamente.
- GET nao deve alterar estado sensivel.
- SameSite ajuda, mas arquitetura e contexto importam.

Exemplo tecnico

Codigo / comando

```
if (req.body.csrfToken !== req.session.csrfToken) {
  return res.sendStatus(403);
}
```

Portugol - equivalente didatico

```
se tokenRecebido <> tokenDaSessao entao
    responder(403)
fimse
```

Aula 10 - Burp Suite: Proxy, Intercept e Repeater

Burp atua entre navegador e aplicacao para observar e repetir requisicoes HTTP. Proxy permite interceptar; Repeater ajuda a modificar e reenviar manualmente uma requisicao. O valor didatico e enxergar o que o navegador realmente manda.

Pontos-chave

- Use apenas em ambientes autorizados.
- Compare requisicao original e resposta antes de formular hipoteses.
- Ferramenta nao substitui raciocinio.

Exemplo tecnico

Codigo / comando

```
// Representacao de uma requisicao
const req = { method: "GET", path: "/profile?id=10" };
```

Portugol - equivalente didatico

```
metodo <- "GET"
caminho <- "/profile?id=10"
escreva(metodo + " " + caminho)
```